

LINUX-BASED HARDWARE HEALTH AND DIAGNOSTIC SYSTEM (HARD_MON)

A Lightweight Linux-Based Hardware Health and Diagnostic System for Real-Time Resource Monitoring, Threshold-Based Alerting, Fault Simulation, and System Health Reporting

INDIVIDUAL PROJECT DOCUMENTATION

Track	Embedded Systems & Linux Kernel Engineering
Name	Himanshu Barik
Registration No.	2341002046
Target Platform	Linux Embedded Systems / POSIX (Ubuntu on WSL2)
Core Standard	ISO C++17
Academic Year	2026-2027
Repository	https://github.com/HimanshuBarik01/Wipro-training
Submission Date	October 2026

1. PROJECT OVERVIEW & PROBLEM STATEMENT

Introduction. Linux-Based Hardware Health and Diagnostic System (HARD_MON) is a Linux-based monitoring and diagnostic application designed to observe important system resources and identify abnormal operating conditions. The system collects resource information from Linux, processes the raw data, evaluates the current condition, and presents a meaningful health state or alert.

Problem Statement. Linux exposes useful system information through kernel interfaces, but raw counters and statistics are not always easy to interpret as an overall health condition. HARD_MON creates a simple monitoring pipeline that converts low-level system information into understandable diagnostic information.

Core Flow

Linux System → Data Collection → Data Processing → Health Analysis → Health Classification → Alert / Diagnostic Output

Objectives

- Collect system resource information from Linux.
- Monitor CPU, memory, and disk-related parameters.
- Process raw system information into usable values.
- Analyze resource conditions and classify system health.
- Generate alerts when a monitored condition becomes concerning.
- Provide a foundation for further hardware and system diagnostics.

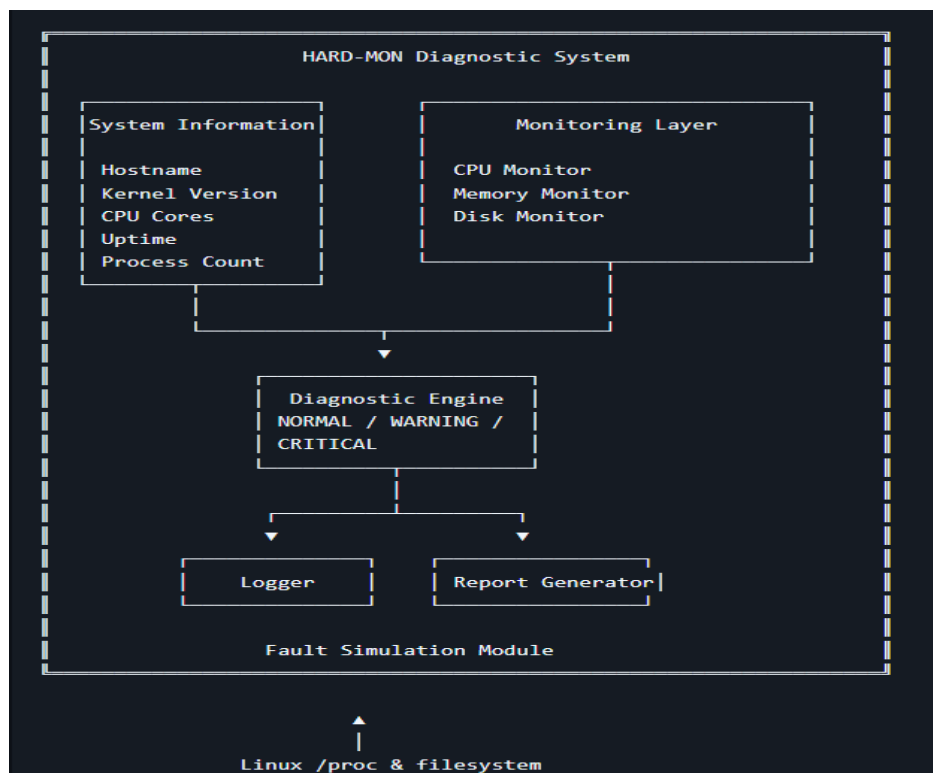
Scope

The current scope covers CPU utilization, memory utilization, disk usage, health classification, and diagnostic/alert output. Future extensions may include temperature, fan, hardware-sensor, historical, remote, or predictive monitoring.

Why this project? The project connects Linux system-level information with a higher-level diagnostic mechanism. It demonstrates practical understanding of Linux interfaces, resource monitoring, data processing, and system-level testing.

2. SYSTEM ARCHITECTURE & WORKING PRINCIPLE

Architecture:-



Working Steps:-

1. **Collect:** Read required system information exposed by Linux.
2. **Process:** Extract and convert raw values into meaningful monitoring parameters.
3. **Analyze:** Evaluate resource values using the project's health criteria.
4. **Classify:** Assign a health state such as NORMAL, WARNING/MEDIUM, or CRITICAL.
5. **Alert:** Provide a diagnostic indication when a resource reaches a concerning condition.

CPU Monitoring

CPU utilization is determined from two CPU counter readings taken at different times. The change between the readings is processed to estimate CPU activity during the observation interval.

First CPU reading → time interval → second CPU reading → counter difference → CPU utilization

Memory Monitoring

Memory information is read from **/proc/meminfo**. The required memory values are extracted and processed to determine current memory utilization before being passed to the health analyzer.

Disk Monitoring

Disk/filesystem usage is collected using the Linux system/filesystem information available to the application. The resulting utilization value is evaluated along with CPU and memory conditions.

3. IMPLEMENTATION & CORE MODULES

Major Modules

Module	Responsibility
System Data Collector	Obtains raw CPU, memory, and disk information from Linux.
CPU Monitor	Reads CPU counters, takes multiple readings, and determines CPU utilization.
Memory Monitor	Reads /proc/meminfo and calculates/derives memory utilization.
Disk Monitor	Obtains disk/filesystem usage information.
Health Analyzer	Evaluates processed resource values against defined health criteria.
Alert / Diagnostic Module	Reports warning or critical conditions to the user.
User Interface / Output	Presents current values and health information in readable form.

Data Flow

Linux Kernel/System Interfaces → Collection → Processing → Utilization Values → Health Analyzer → Health State → Alert / Diagnostic Output

Version Control

Git is used to maintain source-code history and support structured development. Features can be developed and tested separately before integration into the main project.

Example: main → development → CPU / Memory / Disk / Health Analysis / Alert features

4. TESTING, DIAGNOSTICS & VALIDATION

Testing Objective

Testing verifies that HARD_MON correctly collects system information, processes the values, produces reasonable utilization results, classifies system conditions, and generates alerts when required.

Unit and Integration Testing

Component	Test Objective	Result
CPU Monitor	Verify CPU data collection/calculation	Pass
Memory Monitor	Verify /proc/meminfo parsing	Pass
Disk Monitor	Verify disk information	Pass
Health Analyzer	Verify health classification	Pass
Alert Module	Verify alert generation	Pass

System Test Scenarios

Condition	Expected Behavior	Actual
Normal CPU usage	Normal health state	Observed
High CPU usage	Warning/Critical	Not tested
Normal memory	Normal health state	Observed
High memory	Warning/Critical	Not tested
High disk usage	Warning/Critical	Not tested
Multiple resources stressed	Appropriate health status	Not tested

Controlled Stress / Fault Testing

Where applicable, a controlled resource load can be introduced to verify whether the diagnostic logic detects abnormal conditions and produces the expected warning or critical indication. The purpose is controlled validation, not system damage.

Diagnostic output

```
----- HEALTH REPORT -----  
  
CPU Usage   : 0.06 % [NORMAL]  
RAM Usage   : 8.11 % [NORMAL]  
Disk Usage  : 5.42 % [NORMAL]  
  
-----  
Overall Health : NORMAL  
-----
```

5. RESULTS, APPLICATIONS & FUTURE SCOPE

Project Outcome

HARD_MON provides a structured mechanism for converting Linux system information into understandable health and diagnostic information. The final implementation should demonstrate continuous or repeated resource monitoring, health evaluation, and appropriate diagnostic output.

Final Results

Parameter	Observed / Final Result
CPU Monitoring	CPU usage is successfully collected from /proc/stat and displayed/calculated by the system.
Memory Monitoring	Memory information is successfully collected from /proc/meminfo and processed by the system.
Disk Monitoring	Disk usage and available storage information are successfully monitored and displayed.
Health Classification	The system classifies the monitored resources into Normal, Warning, or Critical states based on predefined threshold conditions.
Alert Mechanism	The system is designed to generate alerts when monitored resource usage reaches warning or critical thresholds.
Overall Status	System successfully performs Linux-based hardware health monitoring under normal operating conditions.

Applications

Linux-based embedded systems, development and testing environments, system diagnostics, resource monitoring, hardware/software testing, and edge or industrial Linux systems.

Limitations

- The initial version monitors only the parameters implemented in the project.
- Health thresholds require validation and calibration using actual test results.
- Hardware sensor availability can vary between Linux systems.
- The initial system may focus on current-state monitoring rather than long-term historical analysis.

Future Enhancements

Temperature and fan monitoring; hardware sensor integration; historical data storage; graphical dashboard; configurable thresholds; log-based diagnostics; remote monitoring; and predictive health analysis.

Key Technical Learning

Linux system interfaces • /proc filesystem • CPU utilization calculation • Memory monitoring • Disk monitoring • Data processing • Health classification • Alert generation • System-level debugging • Testing and validation • Git-based development

Conclusion

HARD_MON demonstrates how low-level Linux system information can be collected, processed, and converted into meaningful health and diagnostic information. The project connects Linux system-level concepts with practical monitoring, analysis, alerting, and validation, while providing a foundation for more advanced hardware diagnostics.